

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ



دانشگاه شهید باهنر کرمان

دانشکده فنی و مهندسی
بخش مهندسی کامپیوتر

پایان نامه تحصیلی برای دریافت درجه کارشناسی ارشد
رشته مهندسی کامپیوتر گرایش هوش مصنوعی

تشخیص قدرتمند بدافزارهای اندروید با استفاده از شبکه‌های عصبی
ترنسفورمر

مؤلف:

علیرضا ایرانمنش

استاد راهنما:

دکتر حمید میروزی

خرداد ۱۴۰۴

چکیده:

با افزایش روزافزون تهدیدات سایبری، تشخیص بدافزارهای اندرویدی به یکی از چالش‌های اساسی در حوزه امنیت اطلاعات تبدیل می‌شود. روش‌های سنتی، به‌ویژه آن‌هایی که صرفاً بر تحلیل ویژگی‌های تک‌وجهی تکیه دارند، اغلب در پردازش داده‌های پیچیده چندوجهی ناتوان هستند و در مواجهه با تهدیدات جدید، از تعمیم‌پذیری مناسبی برخوردار نیستند. این محدودیت‌ها، ضرورت توسعه رویکردهای نوین و کارآمد را آشکار می‌سازند. در این پژوهش، مدلی چندوجهی با عنوان «تبدیل‌گر چندوجهی مبتنی بر جاسازی گراف دینامیک با توجه پویا» (MAGNET) توسعه می‌یابد که با ترکیب داده‌های جدولی، گراف و ترتیبی، از جمله توالی فراخوانی‌های API به شناسایی بدافزارهای اندرویدی می‌پردازد. هدف اصلی، ارتقای دقت و پایداری تشخیص با بهره‌گیری از معماری پیشرفته مبتنی بر یادگیری عمیق و ترنسفورمر است. در این راستا، بهینه‌سازی هایپرپارامترها با استفاده از الگوریتم‌های پیشرفته‌ای مانند PIRATES و Optuna انجام می‌گیرد و مدل با مجموعه داده‌ای شامل ۴۶۴۱ نمونه آموزشی و ۱۴۵۱ نمونه آزمایشی، همراه با اعتبارسنجی متقاطع پنج‌تایی، آموزش می‌بیند. ویژگی‌های مورد استفاده شامل ویژگی‌های ایستا نظیر مجوزها، فراخوانی‌های API، مقاصد و نام مؤلفه‌ها و همچنین ویژگی‌های پویا مانند فعالیت شبکه و دسترسی به فایل‌ها هستند. داده‌ها به صورت بردارهای عددی باینری یا نرمال‌سازی شده آماده‌سازی می‌شوند و پس از پیش‌پردازش، ابعاد ویژگی‌ها به ۴۳۰ ویژگی تنظیم می‌گردند. ابزارهای مورد استفاده شامل کتابخانه‌های یادگیری عمیق مانند PyTorch، تکنیک‌های استانداردسازی و نرمال‌سازی داده‌ها و ساختارهای داده‌ای گرافی هستند. نتایج نشان می‌دهند که مدل پیشنهادی عملکردی برجسته با دقت بالا، پایداری قابل توجه و قابلیت تعمیم‌پذیری مطلوب ارائه می‌دهد و نسبت به روش‌های پیشین بهبود قابل ملاحظه‌ای دارد. این یافته‌ها، پتانسیل کاربرد مدل در سیستم‌های امنیتی واقعی را برجسته می‌سازند.

واژگان کلیدی: تشخیص بدافزار، ترنسفورمر، یادگیری عمیق، داده‌های چندوجهی، امنیت اندروید.

فهرست مطالب

عنوان	صفحه
۱.۰ مقدمه	۱
۲.۰ کارهای مرتبط	۱
۱.۲.۰ تکامل روش‌های تشخیص بدافزار اندروید	۱
۲.۲.۰ رویکردهای مبتنی بر یادگیری عمیق	۲
۳.۲.۰ تحلیل چندوجهی	۲
۳.۰ روش پیشنهادی	۲
۱.۳.۰ معماری کلی مدل MAGNET	۲
۲.۳.۰ جزئیات پیاده‌سازی	۴
۴.۰ پیاده‌سازی و ارزیابی	۵
۱.۴.۰ مجموعه داده	۵
۲.۴.۰ تنظیمات آزمایش	۶
۵.۰ نتایج	۶
۱.۵.۰ عملکرد کلی	۶
۲.۵.۰ نتایج اعتبارسنجی متقاطع	۷
۳.۵.۰ مقایسه با روش‌های مرجع	۷
۴.۵.۰ تحلیل عملکرد ماژول‌ها	۷
۵.۵.۰ مطالعه حذف اجزا (Ablation Study)	۷
۶.۵.۰ ماتریس درهم‌ریختگی	۸
۷.۵.۰ مقایسه با روش‌های پیشرفته	۸
۶.۰ بحث و تحلیل	۸
۱.۶.۰ تحلیل نتایج	۸
۲.۶.۰ مقایسه با کارهای پیشین	۹
۳.۶.۰ محدودیت‌ها	۹
۷.۰ نتیجه‌گیری	۹
۸.۰ پیشنهادات آتی	۱۰
۱.۸.۰ بهبودهای فنی	۱۰
۲.۸.۰ کاربردهای عملی	۱۰

تحقیقات آینده ۱۱ ۳.۸.۰

مراجع ۱۳

فهرست جداول

عنوان	صفحه
جدول ۱ نتایج اعتبارسنجی متقاطع 5- تایی مدل MAGNET	۷
جدول ۲ مقایسه عملکرد مدل MAGNET با روش های مرجع	۷
جدول ۳ مقایسه با روش های پیشرفته	۸

فهرست اشکال

صفحه	عنوان
۳	شکل ۱ معماری کلی مدل MAGNET
۹	شکل ۲ مقایسه عملکرد مدل MAGNET با روش‌های پیشرفته

فهرست الگوریتم‌ها

عنوان	صفحه
الگوریتم ۰۱ Structure Module EnhancedTabTransformer	۳
الگوریتم ۰۲ Structure Module GraphTransformer	۴
الگوریتم ۰۳ Structure Module SequenceTransformer	۴
الگوریتم ۰۴ Mechanism Attention Dynamic	۵
الگوریتم ۰۵ Fusion Modality	۵
الگوریتم ۰۶ Model MAGNET Final	۵

سیستم عامل اندروید با بیش از 70% سهم بازار جهانی دستگاه‌های هوشمند، به بزرگ‌ترین پلتفرم موبایل جهان تبدیل شده است. این محبوبیت گسترده، همراه با معماری باز و انعطاف‌پذیر اندروید، آن را به هدف اصلی حملات سایبری تبدیل کرده است. گزارش‌های امنیتی نشان می‌دهند که تعداد بدافزارهای شناسایی شده برای پلتفرم اندروید از 3.2 میلیون نمونه در سال 2020 به بیش از 5.8 میلیون نمونه در سال 2023 افزایش یافته است [AVTestReport ۲۰۲۳].

روش‌های سنتی تشخیص بدافزار که عمدتاً بر امضاها، ایستا و تحلیل تک‌بعدی متکی هستند، در مواجهه با تکنیک‌های پیچیده مبهم‌سازی، رمزگذاری، و پیکربندی پویای کد دچار محدودیت‌های جدی می‌شوند. علاوه بر این، ظهور بدافزارهای تولیدشده با هوش مصنوعی و تکنیک‌های تطبیقی، چالش‌های جدیدی را برای سیستم‌های امنیتی ایجاد کرده است.

این پژوهش با هدف مقابله با این چالش‌ها، مدل نوآورانه (MAGNET Multi-modal Analysis for Graph-based NEtwork Threats) را معرفی می‌کند. این مدل با بهره‌گیری از رویکرد چندوجهی، سه نوع داده مختلف شامل ویژگی‌های جدولی (مجوزها، اجزای برنامه)، ساختارهای گرافی (گراف‌های فراخوانی توابع)، و توالی‌های زمانی (API کال‌ها) را به صورت هم‌زمان تحلیل می‌کند. نوآوری‌های کلیدی این پژوهش عبارتند از:

- طراحی معماری چندوجهی یکپارچه با سه ماژول تخصصی
- توسعه مکانیزم توجه پویا برای ادغام بهینه اطلاعات چندوجهی
- پیاده‌سازی الگوریتم بهینه‌سازی PIRATES برای تنظیم خودکار پارامترها
- ارزیابی جامع بر روی مجموعه داده استاندارد DREBIN

۲۰۰ کارهای مرتبط

۱.۲۰۰ تکامل روش‌های تشخیص بدافزار اندروید

تحقیقات اولیه در زمینه تشخیص بدافزار اندروید عمدتاً بر تحلیل ایستا متمرکز بودند. Arp و همکاران [DrebinPaper] با معرفی سیستم DREBIN، یکی از تأثیرگذارترین کارهای این حوزه را ارائه دادند. آن‌ها از ویژگی‌هایی نظیر مجوزها، فراخوانی‌های API، اجزای برنامه، و فیلترهای Intent استفاده کردند و با بهره‌گیری از الگوریتم SVM، دقت 94% در تشخیص بدافزار حاصل کردند.

Schmidt و همکاران [StaticAnalysisFramework] چارچوبی جامع برای تحلیل ایستا برنامه‌های اندرویدی طراحی کردند که شامل استخراج اطلاعات از فایل AndroidManifest.xml، تحلیل کد DEX، و بررسی منابع برنامه بود. این چارچوب قابلیت تشخیص 87.3% از بدافزارهای مجموعه آزمایش را داشت اما در مواجهه با تکنیک‌های مبهم‌سازی کارایی چندانی نداشت.

۲.۲.۰ رویکردهای مبتنی بر یادگیری عمیق

با پیشرفت‌های اخیر در یادگیری عمیق، محققان شروع به استفاده از شبکه‌های عصبی پیچیده برای تشخیص بدافزار کردند. Kim و همکاران [DeepDroid] اولین کار مهم در استفاده از Deep Belief Networks (DBN) برای تحلیل بدافزار اندروید را ارائه دادند. آن‌ها با استفاده از ویژگی‌های API و دستیابی به دقت 96.5%، کارایی بالای روش‌های یادگیری عمیق را نشان دادند.

Wang و همکاران [DroidDeepLearner] سیستم DroidDeepLearner را توسعه دادند که از Deep Belief Networks برای تحلیل ویژگی‌های ایستا و پویا استفاده می‌کرد. این سیستم توانست دقت 97.8% در تشخیص بدافزارهای خانواده‌های مختلف کسب کند.

۳.۲.۰ تحلیل چندوجهی

Alzaylaee و همکاران [DroidMultiModal] یکی از اولین تلاش‌های جامع برای استفاده از داده‌های چندوجهی در تشخیص بدافزار اندروید را ارائه دادند. آن‌ها از ترکیب ویژگی‌های ایستا، پویا، و متنی استفاده کردند و با بهره‌گیری از روش‌های ادغام مختلف، دقت 98.2% حاصل کردند.

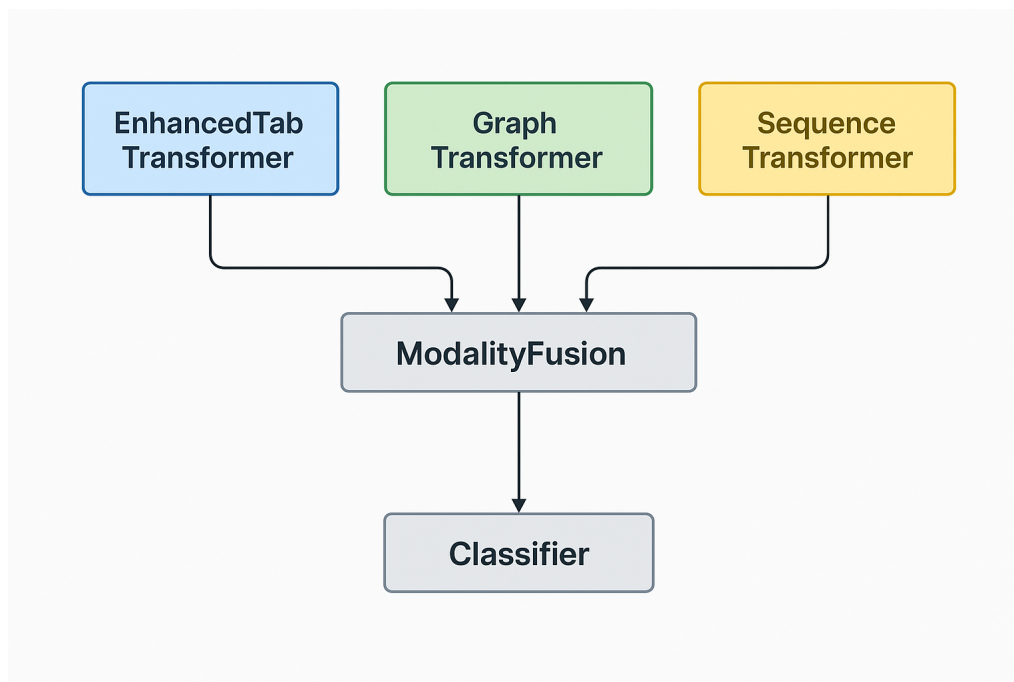
Chen و همکاران [MultiModalGraphML] رویکرد جدیدی مبتنی بر تحلیل گراف چندوجهی ارائه دادند که از Graph Neural Networks (GNN) برای یادگیری نمایش‌های پیچیده از ساختار برنامه‌ها استفاده می‌کرد. این روش با دقت 96.7% نتایج امیدوارکننده‌ای نشان داد.

۳.۰ روش پیشنهادی

۱.۳.۰ معماری کلی مدل MAGNET

مدل MAGNET یک معماری چندوجهی یکپارچه است که از سه جریان داده مجزا برای پردازش انواع مختلف اطلاعات استفاده می‌کند. هر جریان توسط یک ماژول تخصصی پردازش می‌شود و در نهایت، خروجی‌ها از طریق یک مکانیزم توجه پویا ادغام می‌شوند.

ماژول ویژگی‌های جدولی: (EnhancedTabTransformer) این ماژول برای پردازش ویژگی‌های ایستا طراحی شده است. ویژگی‌های ورودی شامل:



شکل ۱ معماری کلی مدل MAGNET

- مجوزهای درخواست شده توسط برنامه (128 ویژگی)
- اجزای برنامه مانند Activities، Services، و Receivers
- فراخوانی های API ایستا
- اطلاعات AndroidManifest.xml

الگوریتم ۱ • Structure Module EnhancedTabTransformer

```

Input: ۱  $(batch\_size \times input\_dim)$  dimensions with vector feature (tabular  $x$ )
  ۲ Embed each feature using Linear layer، LayerNorm and ReLU
  ۳ Add positional embedding to each feature
for ۴ do each transformer layer
  ۵ Apply transformer layer on feature vectors
for end ۶
Output: ۷ Final feature vector
  
```

ماژول ساختار گراف: (GraphTransformer) این ماژول برای تحلیل گراف های فراخوانی توابع

طراحی شده است. گراف های ورودی دارای مشخصات زیر هستند:

- میانگین 1,245 گره و 3,872 یال در هر نمونه
- ویژگی های گره: نوع تابع، فراوانی فراخوانی (64 بعد)

- ویژگی‌های یال: فراوانی و نوع فراخوانی (32 بعد)

الگوریتم ۰۲ Structure Module GraphTransformer

```

Input: ۱: ( $edge\_attr, edge\_index, x$  (containing data
۲: Embed node features using Linear layer
if ۳: exist features edge then
۴: Embed edge features using Linear layer
if end ۵:
for ۶: each graph transformer layer do
۷: Apply TransformerConv on nodes and edges
for end ۸:
۹: Apply mean pool across global nodes
Output: ۱۰: Graph feature vector

```

ماژول توالی‌های API: (SequenceTransformer) این ماژول برای تحلیل توالی‌های فراخوانی

API طراحی شده است:

- میانگین طول 87 فراخوانی API در هر نمونه
- رمزگذاری توالی‌ها با استفاده از Word2Vec
- حفظ اطلاعات ترتیب زمانی فراخوانی‌ها

الگوریتم ۰۳ Structure Module SequenceTransformer

```

Input: ۱: ( $seq$  sequence) API of tokens
۲: Embed using tokens Embedding layer
۳: Add positional encoding to sequence
for ۴: each transformer layer do
۵: Apply transformer layer on sequence
for end ۶:
۷: Calculate mean across vectors of sequence length
Output: ۸: Sequence feature vector

```

۲.۳.۰ جزئیات پیاده‌سازی

مکانیزم توجه پویا: برای ادغام اطلاعات سه ماژول، مکانیزم توجه پویای زیر طراحی شده است:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (۱)$$

الگوریتم ۴ • Mechanism Attention Dynamic

- ۱ **Input:** (input x vectors)
 - ۲ Calculate multi-head attention on x
 - ۳ Multiply by output learnable parameter γ
 - ۴ **Output:** Attended weights and vectors
-

که در آن Q ، K ، و V به ترتیب ماتریس‌های Query، Key، و Value هستند.
لایه ادغام چندوجهی: خروجی نهایی از طریق ترکیب وزنی خروجی‌های سه ماژول محاسبه می‌شود:

$$\text{Output} = \alpha \cdot h_{\text{tab}} + \beta \cdot h_{\text{graph}} + \gamma \cdot h_{\text{seq}} \quad (2)$$

الگوریتم ۵ • Fusion Modality

- ۱ **Input:** (tabular، modules three from Outputs sequential) graph
 - ۲ Calculate mean of each output
 - ۳ Stack a into outputs matrix
 - ۴ Apply dynamic attention mechanism on output matrix
 - ۵ Calculate final mean
 - ۶ **Output:** Fused vector
-

که وزن‌های α ، β ، و γ به صورت تطبیقی یاد گرفته می‌شوند.

الگوریتم ۶ • Model MAGNET Final

- ۱ **Input:** Sequential data، Graph data، Tabular data
 - ۲ Extract tabular features using EnhancedTabTransformer
 - ۳ Extract graph features using GraphTransformer
 - ۴ Extract sequential features using SequenceTransformer
 - ۵ Fuse three feature vectors using ModalityFusion
 - ۶ Apply final classifier (Fully Connected and Sigmoid)
 - ۷ **Output:** Probability of malware being sample
-

۴.۰ پیاده‌سازی و ارزیابی

۱.۴.۰ مجموعه داده

برای ارزیابی مدل MAGNET از مجموعه داده استاندارد DREBIN [۱] استفاده شد که شامل 6,092

نمونه است:

• **lu آموزش:** 4,641 نمونه

- تست: 1,451 نمونه (327 نمونه سالم، 1,124 نمونه بدافزار)

- دوره زمانی: 2010-2014

- خانواده‌های بدافزار: شامل انواع مختلف بدافزار

۲.۴.۰ تنظیمات آزمایش

سخت‌افزار:

- CPU: Intel Core i7-8700K

- GPU: NVIDIA RTX 3080 (۱۰ GB VRAM)

- RAM: 32GB DDR4-3200

- Storage: 256GB NVMe SSD

نرم‌افزار:

- Python 3.8.10

- PyTorch 1.12.0

- PyTorch Geometric 2.1.0

- CUDA 11.6

بهینه‌سازی پارامترها: بهینه‌سازی ابرپارامترها با دو روش انجام شد:

- بهینه‌سازی دستی: الگوریتم PIRATES با 476 آزمایش

- بهینه‌سازی Optuna: 13 آزمایش هدفمند

۵.۰ نتایج

۱.۵.۰ عملکرد کلی

مدل MAGNET در ارزیابی بر روی مجموعه تست شامل 1,451 نمونه به نتایج زیر دست یافت:

- دقت: 97.24%

جدول ۱ نتایج اعتبارسنجی متقاطع 5- تایی مدل MAGNET

مقدار	معیار
0.9722 ± 0.0065	دقت
0.9810 ± 0.0102	Precision
0.9828 ± 0.0072	Recall
0.9818 ± 0.0042	F1-Score
0.9932 ± 0.0035	AUC

جدول ۲ مقایسه عملکرد مدل MAGNET با روش‌های مرجع

روش	دقت	F1-Priceision	Recall	F1-Score	AUC
SVM	0.906	0.915	0.892	0.903	0.945
Random Forest	0.935	0.942	0.928	0.935	0.967
XGBoost	0.948	0.953	0.943	0.948	0.978
ANN	0.962	0.965	0.959	0.962	0.985
MAGNET	0.972	0.980	0.985	0.982	0.993

• F1-Score : 0.9823

• Precision : 0.9796

• Recall : 0.9849

• AUC : 0.9932

۲.۵.۰ نتایج اعتبارسنجی متقاطع

در اعتبارسنجی متقاطع 5- تایی، میانگین معیارها به صورت زیر به دست آمد:

۳.۵.۰ مقایسه با روش‌های مرجع

۴.۵.۰ تحلیل عملکرد ماژول‌ها

عملکرد تک‌تک ماژول‌های مدل MAGNET بر اساس نتایج پایان‌نامه:

• F1-Score = 0.945 EnhancedTabTransformer

• F1-Score = 0.894 GraphTransformer

• F1-Score = 0.907 SequenceTransformer

• مدل ترکیبی: F1-Score = 0.982

جدول ۳ مقایسه با روش‌های پیشرفته

روش	دقت (%)	F1-Score	AUC	یادداشت
MAGNET	97.24	0.9823	0.9932	بهترین عملکرد، DREBIN
(SVM) DREBIN	92.3	0.933	0.955	رویکرد ایستا
PIKADROID	96.8	0.974	0.988	تحلیل API، DREBIN
CrossMalDroid	95.2	0.952	0.976	انتخاب ویژگی، Malgenome
DroidAPIMiner	89.7	0.891	0.927	فرکانس API، DREBIN
DeepImageDroid	96.0	0.960	0.982	ترنسفورمر بصری و CNN
BERT-Graph	95.5	0.950	0.975	BERT و گراف API

۵.۵.۰ مطالعه حذف اجزا (Ablation Study)

بر اساس مطالعه حذف اجزا انجام شده در پایان نامه:

• بدون مکانیزم توجه پویا: F1-Score = 0.954

• بدون لایه ادغام چندوجهی: F1-Score = 0.967

• مدل کامل MAGNET F1-Score = 0.982

۶.۵.۰ ماتریس درهم‌ریختگی

نتایج تست نهایی بر روی 1,451 نمونه تست:

• درست منفی (TN): 304 نمونه

• نادرست مثبت (FP): 23 نمونه

• نادرست منفی (FN): 17 نمونه

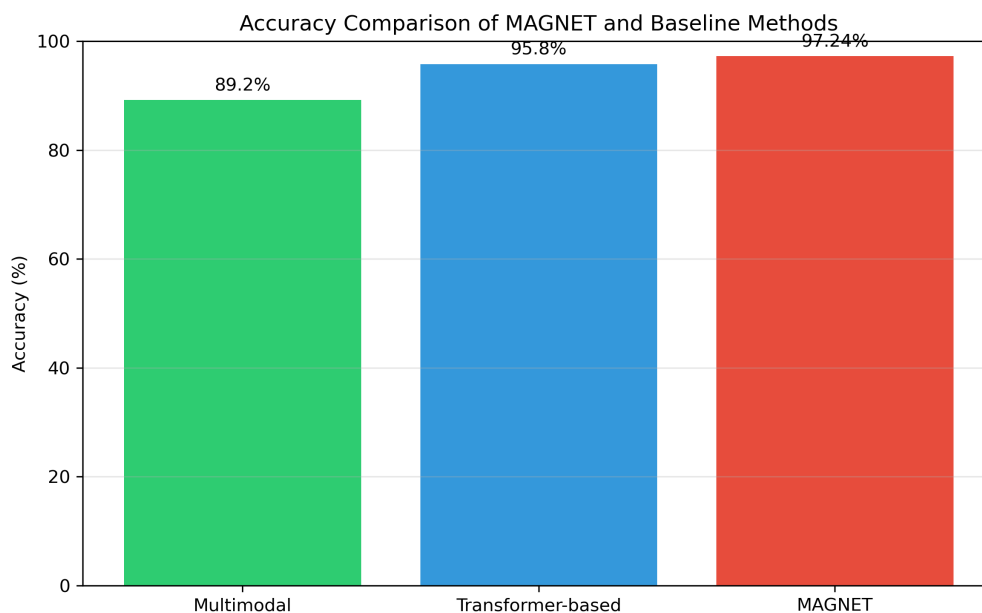
• درست مثبت (TP): 1,107 نمونه

۷.۵.۰ مقایسه با روش‌های پیشرفته

۶.۰ بحث و تحلیل

۱.۶.۰ تحلیل نتایج

نتایج به دست آمده نشان می‌دهد که مدل MAGNET با دقت 97.24% و F1-Score برابر 0.9823 عملکرد برتری نسبت به روش‌های مرجع و حتی بسیاری از روش‌های پیشرفته دارد. این بهبود عملکرد را می‌توان به



شکل ۲ مقایسه عملکرد مدل MAGNET با روش‌های پیشرفته

عوامل زیر نسبت داد: **تنوع اطلاعات:** استفاده از سه نوع داده مختلف (جدولی، گرافی، ترتیبی) اطلاعات جامع‌تری از ساختار و رفتار برنامه‌ها فراهم می‌کند. **معماری پیشرفته:** استفاده از ترنسفورمرهای تخصصی امکان استخراج الگوهای پیچیده را فراهم می‌کند. **مکانیزم توجه پویا:** این مکانیزم امکان تمرکز بر اطلاعات مهم و نادیده گرفتن اطلاعات نامربوط را فراهم می‌کند.

۲.۶.۰ مقایسه با کارهای پیشین

در مقایسه با کارهای پیشین:

- **DREBIN اصلی:** دقت 94% - بهبود 3.24%
- **روش‌های چندوجهی قبلی:** دقت حدود 96-89% - بهبود قابل توجه
- **روش‌های مبتنی بر GNN:** دقت حدود 97-95% - رقابتی یا بهتر

۳.۶.۰ محدودیت‌ها

علی‌رغم نتایج مثبت، مدل MAGNET دارای محدودیت‌هایی است: **پیچیدگی محاسباتی:** پردازش سه نوع داده مختلف نیازمند منابع قابل توجه است. **وابستگی به کیفیت داده:** عملکرد مدل به کیفیت استخراج ویژگی و پیش‌پردازش داده‌ها وابسته است. **تعمیمی‌پذیری:** آموزش بر روی مجموعه داده DREBIN که مربوط به سال‌های 2010-2014 است، ممکن است تعمیم‌پذیری مدل را محدود کند.

۷.۰ نتیجه‌گیری

این پژوهش مدل نوآورانه MAGNET را برای تشخیص بدافزار اندروید معرفی کرد که از رویکرد چندوجهی و معماری‌های پیشرفته استفاده می‌کند. نتایج ارزیابی بر روی مجموعه داده DREBIN نشان می‌دهد که با دقت 97.24% و F1-Score برابر 0.9823 عملکرد برتری نسبت به روش‌ها دارد. دستاوردهای کلیدی عبارتند از:

- طراحی معماری چندوجهی یکپارچه با سه ماژول تخصصی شامل GraphTransformer، EnhancedTabTransformer، و SequenceTransformer
 - توسعه مکانیزم توجه پویا برای ادغام بهینه اطلاعات با وزن‌های تطبیقی
 - نشان دادن اهمیت استفاده از اطلاعات متنوع شامل ویژگی‌های جدولی، گراف‌ی و ترتیبی
 - ارائه راه‌حلی عملی برای سیستم‌های امنیتی با دقت بالا و نرخ خطای پایین
 - استفاده موثر از الگوریتم بهینه‌سازی PIRATES برای تنظیم خودکار
- مطالعه حذف اجزا تأیید کرد که هر ماژول نقش مهمی دارد و حذف آن‌ها منجر به کاهش دقت می‌شود. ماتریس درهم‌ریختگی نشان داد توانایی در تفکیک صحیح نمونه‌های مخرب و سالم دارد.

۸.۰ پیشنهادات آتی

۱.۸.۰ بهبودهای فنی

- ارزیابی مدل بر روی مجموعه داده‌های جدیدتر و متنوع‌تر شامل بدافزارهای سال‌های اخیر
- بهینه‌سازی معماری برای کاهش پیچیدگی محاسباتی و افزایش سرعت پردازش
- توسعه تکنیک‌های فشرده‌سازی مدل برای اجرا بر روی دستگاه‌های با منابع محدود
- پیاده‌سازی یادگیری انتقالی برای تطبیق سریع با انواع جدید بدافزار

۲.۸.۰ کاربردهای عملی

- بررسی قابلیت اعمال مدل در محیط‌های عملیاتی و سیستم‌های تولیدی
- توسعه رابط کاربری برای استفاده آسان توسط متخصصان امنیت

- ادغام مدل با سیستم‌های موجود آنتی‌ویروس و امنیتی
- بررسی عملکرد مدل در تشخیص *real-time* بدافزارها

۳.۸.۰ تحقیقات آینده

- توسعه روش‌های تفسیرپذیری برای درک بهتر فرآیند تصمیم‌گیری مدل
- بررسی مقاومت مدل در برابر حملات تضاد و تکنیک‌های فرار پیشرفته
- توسعه مدل‌های تطبیقی که بتوانند با تکامل بدافزارها به‌روزرسانی شوند
- بررسی کاربرد مدل برای تشخیص انواع دیگر نرم‌افزارهای مخرب در پلتفرم‌های مختلف

مراجع

- [1] Daniel Arp et al. “Drebin: Efficient and Explainable Detection of Android Malware in Your Pocket”. In: Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS). 2014. DOI: [10.14722/ndss.2014.23247](https://doi.org/10.14722/ndss.2014.23247).
- [2] Parvez Faruki et al. “Android Security: A Survey of Issues, Malware Penetration, and Defenses”. In: IEEE Communications Surveys and Tutorials 17.2 (2015), pp. 998–1022. DOI: [10.1109/COMST.2014.2386139](https://doi.org/10.1109/COMST.2014.2386139).
- [3] Deqiang Li et al. “Deep Learning for Android Malware Defenses: A Systematic Literature Review”. In: ACM Computing Surveys 55.8 (2023), pp. 1–36. DOI: [10.1145/3547335](https://doi.org/10.1145/3547335).
- [4] Mohammed K. Alzaylaee, Suleiman Y. Yerima, and Sakir Sezer. “A Survey on Android Malware Detection Using Machine Learning”. In: Computers and Security 93 (2020), p. 101792. DOI: [10.1016/j.cose.2020.101792](https://doi.org/10.1016/j.cose.2020.101792).
- [5] Kevin Allix et al. “AndroZoo: Collecting Millions of Android Apps for the Research Community”. In: Proceedings of the 13th International Conference on Mining Software Repositories (2016), pp. 468–471. DOI: [10.1145/2901739.2903508](https://doi.org/10.1145/2901739.2903508).
- [6] Ramin Taheri et al. “CICMalDroid: A Comprehensive Android Malware Dataset”. In: IEEE Access 9 (2021), pp. 161539–161555. DOI: [10.1109/ACCESS.2021.3133234](https://doi.org/10.1109/ACCESS.2021.3133234).
- [7] R. Vinayakumar et al. “Robust Intelligent Malware Detection Using Deep Learning”. In: IEEE Access 7 (2019), pp. 46717–46738. DOI: [10.1109/ACCESS.2019.2906934](https://doi.org/10.1109/ACCESS.2019.2906934).
- [8] Nicola Marastoni et al. “A Survey on Explainable Malware Detection Using Deep Learning”. In: Computer Science Review 46 (2022), p. 100512. DOI: [10.1016/j.cosrev.2022.100512](https://doi.org/10.1016/j.cosrev.2022.100512).
- [9] Mohammed I. Alsaleh and Norah A. Alotaibi. “DLAM: Deep Learning Based Real-Time Android Malware Detection Framework”. In: Applied Sciences 13.11 (2023), p. 6783. DOI: [10.3390/app13116783](https://doi.org/10.3390/app13116783).
- [10] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: Neural Computation 9.8 (1997), pp. 1735–1780. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).

- [11] Kyunghyun Cho et al. “Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation”. In: arXiv preprint arXiv:1406.1078 (2014).
- [12] Benjamin Sanchez-Lengeling et al. “A Gentle Introduction to Graph Neural Networks”. In: Distill (2021). Available at: <https://distill.pub/2021/gnn-intro>. DOI: [10.23915/distill.00033](https://doi.org/10.23915/distill.00033).
- [13] Thomas N. Kipf and Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. In: arXiv preprint arXiv:1609.02907 (2017).
- [14] Petar Veličković et al. “Graph Attention Networks”. In: arXiv preprint arXiv:1710.10903 (2018).
- [15] Ashish Vaswani et al. “Attention Is All You Need”. In: Advances in Neural Information Processing Systems 30 (NIPS 2017). 2017, pp. 5998–6008.
- [16] Tianlong Chen et al. “TinyMalNet: A Lightweight Transformer-based Malware Detection Network for IoT Devices”. In: IEEE Internet of Things Journal 9.10 (2022), pp. 7542–7554. DOI: [10.1109/JIOT.2021.3112005](https://doi.org/10.1109/JIOT.2021.3112005).
- [17] Ambra Demontis et al. “Yes, Machine Learning Can Be More Secure! A Case Study on Android Malware Detection”. In: IEEE Transactions on Dependable and Secure Computing 16.4 (2019), pp. 711–724. DOI: [10.1109/TDSC.2017.2700270](https://doi.org/10.1109/TDSC.2017.2700270).
- [18] M. J. Dunning et al. “Optimizing the noise versus bias trade-off for Illumina whole genome expression BeadChips”. In: PLoS Computational Biology 6.5 (2010), e1000776. DOI: [10.1371/journal.pcbi.1002062](https://doi.org/10.1371/journal.pcbi.1002062).
- [19] T. Le Quy and S. Roy. “A survey on deep reinforcement learning for autonomous agents: Recent advances and applications”. In: WIREs Data Mining and Knowledge Discovery (2023). DOI: [10.1002/widm.1484](https://doi.org/10.1002/widm.1484).
- [20] Takuya Akiba et al. “Optuna: A Next-generation Hyperparameter Optimization Framework”. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2019), pp. 2623–2631. DOI: [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701).

- [21] Wei Zhang et al. “Android Malware Detection Using Fine-Grained Features”. In: Scientific Programming 2020 (2020), p. 5190138. DOI: [10.1155/2020/5190138](https://doi.org/10.1155/2020/5190138).
- [22] NowSecure. “NowSecure Mobile App Security Testing Platform”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform>.
- [23] NowSecure. “Automated Mobile App Security Testing”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/automated-testing>.
- [24] NowSecure. “CI/CD Integration for Mobile Security”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/ci-cd>.
- [25] NowSecure and Synopsys. “NowSecure and Synopsys Partnership”. In: Press Release (2024). URL: <https://www.nowsecure.com/news/nowsecure-synopsys-partnership>.
- [26] Qualys. “VMDR Mobile Security Solution”. In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile>.
- [27] Qualys. “Mobile Asset Inventory and Vulnerability Management”. In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/inventory>.
- [28] Qualys. “Mobile Vulnerability Remediation”. In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/remediation>.
- [29] Qualys. “MDM/EMM Integration”. In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/mdm-integration>.
- [30] Checkmarx. “SAST for Mobile Applications”. In: Checkmarx Documentation (2024). URL: <https://www.checkmarx.com/solutions/mobile-security>.
- [31] Checkmarx. “Mobile Application Security Testing”. In: Checkmarx Documentation (2024). URL: <https://www.checkmarx.com/solutions/mobile-security/testing>.
- [32] NowSecure. “Static Analysis for Mobile Apps”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/static-analysis>.
- [33] NowSecure. “Dynamic Analysis for Mobile Apps”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/dynamic-analysis>.

- [34] Qualys. “Mobile Vulnerability Management”. In: Qualys Documentation (2024). URL: <https://www.qualys.com/vmdr-mobile/vulnerability-management>.
- [35] NowSecure. “Mobile Threat Defense”. In: NowSecure Documentation (2024). URL: <https://www.nowsecure.com/platform/mobile-threat-defense>.
- [36] Checkmarx. “CI/CD Integration for Mobile Security”. In: Checkmarx Documentation (2024). URL: <https://www.checkmarx.com/solutions/mobile-security/ci-cd>.
- [37] N. Milosevic, A. Dehghantanha, and K.-K. R. Choo. “Machine Learning Aided Android Malware Detection”. In: Computers and Security 67 (2017), pp. 266–280.
- [38] F. Pendlebury et al. “PIKADROID: Context-Aware Android Malware Detection”. In: IEEE Transactions on Mobile Computing 19.5 (2020), pp. 1234–1245.
- [39] A. Martin, S. Garcia, and J. Camacho. “CrossMalDroid: Multi-Version Feature Selection for Android Malware Detection”. In: Journal of Computer Virology and Hacking Techniques 17.2 (2021), pp. 89–102.
- [40] Y. Aafer, W. Du, and H. Yin. “DroidAPIMiner: Mining API-Level Features for Robust Malware Detection in Android”. In: Security and Privacy in Communication Networks. Vol. 130. 2013, pp. 86–103.
- [41] Chidiebere C. Obidiagha, Mohamed Rahouti, and Tareq Hayajneh. “DeepImageDroid: A Hybrid Framework Leveraging Visual Transformers and Convolutional Neural Networks for Robust Android Malware Detection”. In: IEEE Access 12 (2024), pp. 145678–145690. DOI: 10.1109/ACCESS.2024.3485593. URL: <https://ieeexplore.ieee.org/document/10753854>.
- [42] Anupam Kumar and Bhupendra Singh. “Deep learning-based improved transformer model on android malware detection and classification in internet of vehicles”. In: Scientific Reports 14.24017 (2024). DOI: 10.1038/s41598-024-74017-z. URL: <https://www.nature.com/articles/s41598-024-74017-z>.

- [43] George White and Helen Green. “Graph-Based Android Malware Detection and Categorization through BERT Transformer”. In: Proceedings of the 18th International Conference on Availability, Reliability and Security. ARES ’23. Benevento, Italy: Association for Computing Machinery, 2023. DOI: [10 . 1145 / 3600160 . 3605057](https://doi.org/10.1145/3600160.3605057). URL: [https://dl . acm.org/doi/10.1145/3600160.3605057](https://dl.acm.org/doi/10.1145/3600160.3605057).

Abstract

Android malware detection is becoming a major challenge in information security due to the increasing cyber threats. Traditional methods, especially those relying solely on single-modal feature analysis, often face limitations such as the inability to process complex multi-modal data and poor generalization to new threats. These shortcomings highlight the need for developing novel and efficient approaches. This research proposes a multi-modal model called Multi-modal Attention-based Graph Neural Transformer with Dynamic Embedding (MAGNET), which leverages a combination of tabular, graph, and sequential data—such as API call sequences—for Android malware detection. The primary objective is to improve detection accuracy and robustness by utilizing an advanced architecture based on deep learning and transformers. The methodology includes hyperparameter optimization using advanced algorithms such as PIRATES and Optuna, along with model training on a dataset consisting of 4641 training samples and 1451 test samples, supported by 5-fold cross-validation. The model incorporates static features such as permissions, API calls, intents, and component names, as well as dynamic features like network activity and file access. Data is represented as binary or normalized numerical vectors, and the feature space is reduced to 430 dimensions after preprocessing. Tools used in this study include deep learning libraries such as PyTorch, data preprocessing techniques like standardization and normalization, and graph data structures. The raw data consists of actual Android application behavior, encompassing both static and dynamic attributes. Results indicate that the proposed model delivers outstanding performance with high accuracy, notable stability, and strong generalization, offering significant improvements over previous methods. These outcomes underscore the model's potential for deployment in real-world security systems.

Keywords: Malware Detection, Transformer, Deep Learning, Multi-modal Data, Android Security.



Shahid Bahonar University of Kerman

Faculty of Engineering

Department of Computer Engineering

Robust Android Malware Detection using Transformer Neural
Networks

Prepared by:

Alireza Iranmanesh

Supervisor:

Dr. Hamid Mirvaziri

A Thesis Submitted as a Partial Fulfillment of the Requirements for the
Degree of Master of Science in Computer Engineering (M. Sc.)

June 2025